

R3.04 – Qualité de développement

1 – Introduction

Sébastien Thon

BUT Informatique 2ème année
IUT d'Aix-Marseille Université, site d'Arles
Département Informatique

Introduction

Quel objectif pour cette ressource ?

L'objectif de cette ressource est d'approfondir la qualité de développement.

Quels savoirs de référence à étudier ?

- Approfondissement des concepts de développement orienté objet
- Compréhension et mise en œuvre de patrons de conception
- Rédaction de la documentation du code
- Structuration de l'application

Introduction



Nous allons utiliser le langage de programmation orienté objet Java

Crée par SUN Microsystems en 1995

SUN racheté par Oracle Corporation en 2009

Download :

<https://www.oracle.com/java/technologies/downloads/>

Documentation :

<https://docs.oracle.com/en/java/javase/>

Documentation des packages de base :

<https://docs.oracle.com/en/java/javase/20/index.html>

Java est très utilisé dans l'industrie du fait de ses qualités :

- 1) Langage orienté objet**
- 2) Portabilité**
- 3) Robustesse**

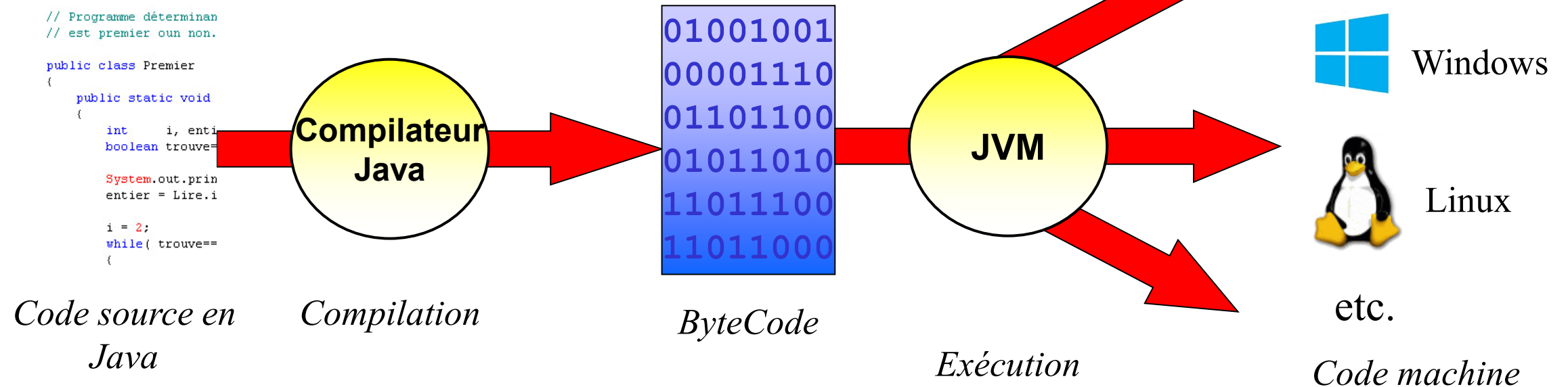
2) Portabilité

Un programme réalisé en Java peut fonctionner sans recompilation sur n'importe quel OS (Windows, Linux, MacOS, etc.)

Cette portabilité est due au fait que le code source Java est compilé pour une machine dite **virtuelle** (appelée **JVM** : Java **V**irtual **M**achine).

Le code machine résultant est nommé **ByteCode**.

Lors de l'**exécution** du programme, ce bytecode sera **interprété par la JVM** → transformation en un code machine compréhensible par le microprocesseur sur lequel tourne l'application.



Example devices powered by Oracle



Small

- RFID Readers
- Parking Meters
- Intelligent Power Module
- Smart Meters



Medium

- Routers & Switches
- Storage Appliances
- Network Management Systems
- Factory Automation Systems
- Security Systems



Large

- Multi Function Printers
- ATMs
- POS Systems
- In-Flight Entertainment Systems
- Electronic Voting Systems
- Medical Imaging Systems



Une application est écrite une fois pour toute en Java et fonctionnera sur n'importe quelle machine dotée d'une JVM.

→ Réduction très importante des coûts de développement : pas besoin de réécrire le code source de l'application ou de le recompiler pour plusieurs systèmes différents.



3) Robustesse

Java = langage « **robuste** » :

- Langage fortement typé → permet moins de "bidouillages" que le langage C, par exemple.
- Détection de bugs classiques de programmes C et C++ comme le dépassement de tableau.
- Contrairement à C et C++, ne permet pas l'usage de pointeurs pour directement adresser des portions de mémoire.
- Gestion « propre » de la mémoire (désallocation automatique).
- ...

Un exemple de programme

```
import java.util.*;

public class Bonjour
{
    public static void main (String [] arg)
    {
        String nom;
        int age;
        Scanner clavier = new Scanner(System.in);

        System.out.println("Entrez votre nom : ");
        nom = clavier.nextLine();

        System.out.println("Entrez votre age : ");
        age = clavier.nextInt();

        System.out.println("Vous vous appelez " + nom
                           + " et vous avez " + age + " ans");
    }
}
```

Remarques

```
public class Bonjour
```

Un programme peut être réparti en plusieurs **classes**, stockées dans des fichiers différents.

Le programme doit être tapé dans un fichier d'extension **.java** portant le même nom que la classe. Par exemple, ici : **Bonjour.java**

Tout le reste du programme est compris entre une accolade ouvrante '{' et une accolade fermante '}'. Il n'y a pas ';' à la fin comme en C++.

Remarques (suite)

```
public static void main (String [] arg)
```

Une classe peut être décomposée en plusieurs **méthodes**.

Dans la classe principale du programme, il faut impérativement une méthode **main()**, qui sera la première à être exécutée (*main* = « principal »).

main() reçoit en paramètre un tableau de chaînes de caractères correspondant aux **arguments** passés au programme sur la ligne de commande lors de l'exécution.

Compilation

Il faut compiler le code source du fichier **Bonjour.java** pour obtenir le fichier de bytecode (code machine pour une machine virtuelle).

On utilise la commande **javac** :

```
javac Bonjour.java
```

→ On obtient un fichier **Bonjour.class** contenant le bytecode.

Exécution

Pour exécuter le fichier de bytecode au moyen de la JVM, on utilise la commande **java** :

```
java Bonjour
```

Remarques :

On ne spécifie pas l'extension **.class**
Attention aux majuscules.

Passage d'arguments lors de l'exécution

Lors de l'exécution avec la commande **java**, on peut passer des arguments au programme :

```
java Bonjour chaine1 1234 chaine2
```

Ces 3 arguments sont transmis automatiquement à la méthode **main** sous la forme d'un tableau de chaînes de caractères :

```
public static void main (String [] args)
```

Le tableau **args** contiendra les 3 chaînes suivantes :

```
"chaine1"    "1234"    "chaine2"
```

Attention, contrairement au C et C++, le premier élément du tableau ne contient pas le nom du programme.

Cas d'un projet composé de plusieurs fichiers

Pour compiler un projet composé de **principal.java**, **compte.java** et de **banque.java** :

```
javac principal.java compte.java banque.java
```

→ crée les fichiers **principal.class**, **compte.class** et **banque.class**

Si **principal.java** contient la fonction **main**, on exécute le programme résultant avec la commande :

```
java principal
```

Autre possibilité

Ecrire un fichier texte (par exemple : projet.txt) dans lequel vous écrivez la liste des noms des fichiers de votre projet :

projet.txt

```
principal.java  
compte.java  
banque.java
```

On compile ensuite avec javac en précisant le nom de ce fichier texte précédé de @ :

```
javac @projet.txt
```


Autre possibilité

Il est aussi possible d'utiliser :

- un système de gestion de projet (Ant, Maven, ...)
- un IDE (*Integrated Development Environment*) = Environnement de développement intégré.

Permet de gérer des projets comportant plusieurs fichiers, d'éditer du code, de compiler, de déboguer, de construire visuellement une interface graphique, etc. (Netbeans, Eclipse)